

프로그래밍 언어 간 의미 거리의 정량적 측정 방법

2026.09.23

한상곤(부산대학교 대학원 정보융합공학과)

연구 배경: 프로그래밍 언어 간 의미 거리를 어떻게 측정할까?

- LM을 활용한 소스 코드 변환은 개발 생산성을 높이는 기술[1]로 많은 곳에서 해당 방법을 활용하고 있으나, 계산 모델이 다른 언어 간 번역은 단순히 구문(syntax)을 치환하는 작업이 아니라 상태 관리, 제어 흐름, 메모리 모델 등 문제 해결 방식을 의미론에 맞게 재구성해야 함.
- 본 연구에서는 트랜스파일 기법을 활용하여 계산 모델 간 변환 난이도를 객관적으로 평가할 수 있는 지표를 제안

제안 방법: 왕복 번역과 근사 고정점[3]

$$C_0 \xrightarrow{1 \text{ hop}} L_1 \xrightarrow{1 \text{ hop}} C_1 \longrightarrow L_2 \longrightarrow C_2 \longrightarrow \dots$$

1. C++ 코드 C_k 를 대상 언어 L 로 번역한 뒤 C++ 코드 C_{k+1} 로 복원
2. 기능 보존 여부를 확인하고, 인접한 복원 코드의 유사도를 JPlag로 측정
3. 기능 실패 없이 $\text{Sim}(C_k, C_{k+1}) \geq 0.85$ 이면 근사 고정점 도달로 판정

측정 지표

$$P_s = \frac{n}{N}, \quad \overline{H} = \frac{1}{n} \sum_{i=1}^n H_i, \quad D = \frac{\overline{H}}{P_s}$$

- 성공 시 필요한 번역이 많고 도달률이 낮을수록 거리가 커진다.

지표	의미
도달률 P_s	전체 유효 시도 N 중 근사 고정점 도달 성공 n 의 비율
평균 홉 수 $\overline{H}$	성공한 시도에서 근사 고정점에 도달하기까지의 평균 편도 번역 횟수
의미 거리 D	평균 홉 수를 근사 고정점 도달률로 보정한 지수

실험 설계

- 추론 토큰 생성을 억제하고, 동일한 지표로 경로별 결과를 비교한다.

구분	설정
기준 언어	C++: 절차형 기반의 객체지향 언어
경유 언어	Python, Java / Haskell(함수형) / Prolog(논리형)
실험 대상	알고리즘 문제 15개 정답 프로그램
반복 횟수	독립적으로 5회 반복 → 경로별 유효 시도 75회
번역 모델	GLM-5.3-Flash
생성 설정	temperature = 0.6 · top_p = 0.95 · max_tokens = 16384

실험 결과

- Python, Java 경로는 도달률과 코드 유사도가 상대적으로 높음
- Haskell, Prolog 경로는 도달률이 낮고, 보정 거리가 크게 나타남

경유 언어	성공/유효	도달률	평균 홑 수	유사도 중앙값	거리 D
Python	62/75	82.7%	5.694	0.64	6.89
Java	56/75	74.7%	5.357	0.62	7.17
Haskell	13/75	17.3%	6.385	0.24	36.83
Prolog	2/75	2.7%	6.000	0.16	225.00

결론 및 논의

- RTT의 평균 홉 수와 근사 고정점 도달률을 결합해 언어 간 의미 거리 지표를 제안
 - 함수형, 논리형 경로의 낮은 도달률과 큰 거리는 계산 모델 차이에 따른 재구성의 어려움을 시사
 - 다중 언어 개발과 프로그램 마이그레이션에서 변환 경로를 비교하는 근거로 활용할 수 있음
- 한계점
 - C++ 기준, 15개 문제, 단일 모델의 결과
 - 문제 특성과 반복 번역의 오류 누적을 함께 고려해야 함
- 논의
 - 하노이탑 문제에서는 Haskell과 Prolog 경유 경로에서도 초기 2회 왕복까지 기능이 보존
 - 재귀 구조가 명확하고 상태 변경이 제한된 문제에서는 초기 단계의 변환이 가능할 수 있음을 보여줌